It's not ideal to have our database password in the docker-compose.yml file: this file should be committed to version control, but it's a security risk to commit files containing secrets. We'll revisit this shortly. Also, technically, we didn't need to set POSTGRES_USER since we're setting its default value. However, I've included it because it's good practice to make things configurable.[5]

OK. With our docker-compose.yml updated, we can start Postgres:

```
$ docker-compose up -d database
```

We're starting our database service in detached mode. We can verify it's up and running with:

```
$ docker-compose ps
      Name                Command               State     Ports
-------------------------------------------------------------------------------
myapp_database_1  docker-entrypoint.sh pos…   Up     5432/tcp
myapp_redis_1     docker-entrypoint.sh red…   Up     6379/tcp
myapp_web_1       bin/rails s -b 0.0.0.0      Up     0.0.0.0:3000->3000/tcp
```

We now have three containers for our app, and we can see that our new addition—the database—is running.

As a further check, we can view the database container's output:

```
$ docker-compose logs database
Attaching to myapp_database_1
«...»
database_1  | PostgreSQL init process complete; ready for start up.
database_1  |
database_1  | 2019-01-15 10:07:29.394 UTC [1] LOG: listening on IPv4 address
"0.0.0.0", port 5432
database_1  | 2019-01-15 10:07:29.394 UTC [1] LOG: listening on IPv6 address
"::", port 5432
database_1  | 2019-01-15 10:07:29.397 UTC [1] LOG: listening on Unix socket
"/var/run/postgresql/.s.PGSQL.5432"
database_1  | 2019-01-15 10:07:29.409 UTC [60] LOG:  database system was shut
down at 2019-01-15 10:07:29 UTC
database_1  | 2019-01-15 10:07:29.414 UTC [1] LOG:  database system is ready
to accept connections
```

Remember that this command shows the *container's* logs—that is, its output rather than Postgres' log file output.

5. https://12factor.net/config